

Questão 3-

a) Por que esse código é mais difícil de analisar do que os anteriores?

Porque a inversão e o aninhamento na ordem de aquisição dos recursos não estão explícitos em uma única função. O risco está escondido (abstraído) por chamadas de funções auxiliares (*atualizar_banco* chama *registrar_log*). Visualmente, olhando apenas para o escopo de *thread1*, não parece haver um conflito direto de chaves, o que exige que o desenvolvedor rastreie toda a pilha de chamadas (*call stack*) para detectar o perigo.

b) Pode ocorrer impasse? Justifique.

Sim, ocorre um auto-bloqueio (impasse de thread única). A *thread1* adquire *log_mutex*, entra em *atualizar_banco()* onde adquire *banco_mutex*, e depois entra em *registrar_log()*, onde tenta readquirir *log_mutex* sem tê-lo liberado. Como o mutex padrão é não-reentrante, a thread congela esperando sua própria liberação.

c) Proponha uma segunda thread que complete o cenário de deadlock.

```
void* thread2(void* arg){  
    //Tenta acessar o atualizar o banco primeiro.  
    atualizar_banco();  
    return NULL;  
}
```

d) Reescreva a solução em C de modo a evitar o problema

Solução no arquivo *resolucao3.c*